

PROJECT DOCUMENTATION

Student Record and Academic Management System

Course: Computational Thinking and Basic Programming

Student: Pranav Mohachakar

Program: B.Tech AI-ML

Project Type: Console-Based Python Application

1. Problem Statement

Managing student academic information manually can make searching records, calculating averages, and determining grades slow and error-prone. This project develops a simple console-based Student Record and Academic Management System that stores student information and provides basic academic operations through a menu-driven Python program.

2. Objective

The objective is to apply computational thinking and a structured software-development process before and during programming. The project includes problem analysis, requirements, algorithms, flowcharts/pseudocode planning, Python implementation, testing, and documentation.

3. Requirement Analysis

Functional Requirements

- Add student information
- Display student records
- Search student details using roll number
- Calculate and display average marks
- Display grades
- Exit the application

Non-Functional Requirements

- Simplicity
- Ease of use
- Accuracy
- Reliability

4. Data Stored for Each Student

The program stores the student's name, roll number, department, semester, subject marks, attendance percentage, calculated average, and grade.

5. Algorithms

Algorithm: Add Student

1. Start the Add Student function.
2. Accept name, roll number, department and semester.
3. Accept marks and attendance.
4. Calculate the average marks.
5. Calculate the grade.
6. Store the information in a dictionary.
7. Add the dictionary to the student list.
8. Display a success message.

Algorithm: Search Student

1. Accept a roll number.
2. Traverse the student list.
3. Compare each stored roll number with the entered roll number.
4. If matched, display the student details.

5. Otherwise display 'Student not found'.

Algorithm: Grade Calculation

1. Receive average marks.
2. Compare the average with grade ranges using if/elif conditions.
3. Return A+, A, B, C, D, or F.

Algorithm: Display Records

1. Check whether the student list is empty.
2. If empty, display a message.
3. Otherwise traverse the list and display each student's details.

6. Flowchart Logic

Main Menu: Start → Display Menu → Enter Choice → Perform Selected Operation → Return to Menu → Exit → Stop

Add Student: Start → Input Student Details → Input Marks → Calculate Average → Calculate Grade → Store Record → Display Success Message → End

Search Student: Start → Enter Roll Number → Search Records → Record Found? → Yes: Display Details / No: Display Not Found → End

Grade Calculation: Start → Input Average → Check Grade Range → Assign Grade → Return Grade → End

7. Pseudocode

START

Create empty STUDENTS list

REPEAT

Display menu

Read CHOICE

IF CHOICE = 1

Read student information and marks

Calculate average

Calculate grade

Store record

ELSE IF CHOICE = 2

Display all records

ELSE IF CHOICE = 3

Read roll number and search student

ELSE IF CHOICE = 4

Display average

ELSE IF CHOICE = 5

Display grade

ELSE IF CHOICE = 6

Exit program

ELSE

Display invalid choice

UNTIL CHOICE = 6

STOP

8. Python Concepts Used

Variables, data types, user input, formatted output, conditional statements, loops, functions, lists, dictionaries, and a menu-driven program are used in the implementation.

9. Source Code

```
students = []  
def calculate_grade(average):  
    if average >= 90:
```

```

        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    else:
        return "F"

def add_student():
    name = input("Enter student name: ")
    roll = input("Enter roll number: ")
    department = input("Enter department: ")
    semester = input("Enter semester: ")
    python_marks = float(input("Enter Python marks: "))
    maths_marks = float(input("Enter Maths marks: "))
    chemistry_marks = float(input("Enter Chemistry marks: "))
    attendance = float(input("Enter attendance percentage: "))

    average = (python_marks + maths_marks + chemistry_marks) / 3
    grade = calculate_grade(average)

    student = {
        "name": name, "roll": roll, "department": department,
        "semester": semester, "python": python_marks,
        "maths": maths_marks, "chemistry": chemistry_marks,
        "attendance": attendance, "average": average, "grade": grade
    }
    students.append(student)
    print("Student record added successfully!")

def display_students():
    if len(students) == 0:
        print("No student records available.")
        return
    for student in students:
        print(student)

def search_student():
    roll = input("Enter roll number to search: ")
    for student in students:
        if student["roll"] == roll:
            print(student)
            return
    print("Student not found.")

def display_average():
    roll = input("Enter roll number: ")
    for student in students:
        if student["roll"] == roll:
            print("Average Marks:", round(student["average"], 2))
            return
    print("Student not found.")

def display_grade():
    roll = input("Enter roll number: ")
    for student in students:
        if student["roll"] == roll:
            print("Grade:", student["grade"])
            return
    print("Student not found.")

while True:
    print("\n1. Add Student")
    print("2. Display Student Records")
    print("3. Search Student")
    print("4. Calculate / Display Average")
    print("5. Display Grade")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ")
    if choice == "1":
        add_student()
    elif choice == "2":
        display_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        display_average()
    elif choice == "5":

```

```
        display_grade()
    elif choice == "6":
        print("Program closed successfully.")
        break
    else:
        print("Invalid choice. Please enter 1 to 6.")
```

10. Testing Report

Test	Input / Action	Expected Result
Add Student	Enter valid student details	Record is added successfully
Display Records	Choose option 2	Stored student records are displayed
Search	Enter existing roll number	Matching student is displayed
Invalid Search	Enter unknown roll number	Student not found message
Average	Choose option 4 and valid roll	Correct average is displayed
Grade	Choose option 5 and valid roll	Calculated grade is displayed
Invalid Menu	Enter choice outside 1-6	Invalid choice message
Exit	Choose option 6	Program closes successfully

11. Screenshots

Add screenshots after running the program. Recommended screenshots: main menu, adding a student, displaying records, searching a student, average/grade output, and exit output.

12. GitHub Repository

The repository should contain the Python source code, this project documentation, screenshots, and a README.md file. After uploading, submit the public repository link provided by GitHub.

13. Learning Reflection

Through this project, I learned how to understand a problem before writing code and how to convert requirements into algorithms, pseudocode, and a working Python program. I practiced variables, conditions, loops, functions, lists, dictionaries, input/output, testing, and debugging. I also learned how a menu-driven program organizes multiple operations and how GitHub can be used to store and share project files.

14. Conclusion

The Student Record and Academic Management System demonstrates the basic stages of software development in a simple console application. It can add, display and search student records, calculate average marks and grades, and manage the operations through a menu. The project focuses on logical reasoning and computational thinking rather than advanced application features.